

主専攻実験第12回

機械学習と深層学習

- 機械学習の基本
 - 数理モデル
 - 損失関数と最適化
- ニューラルネットと深層学習
 - ニューロンと人工ニューロン
 - 畳み込みとV1
 - プーリング
 - 損失関数の設計と正則化



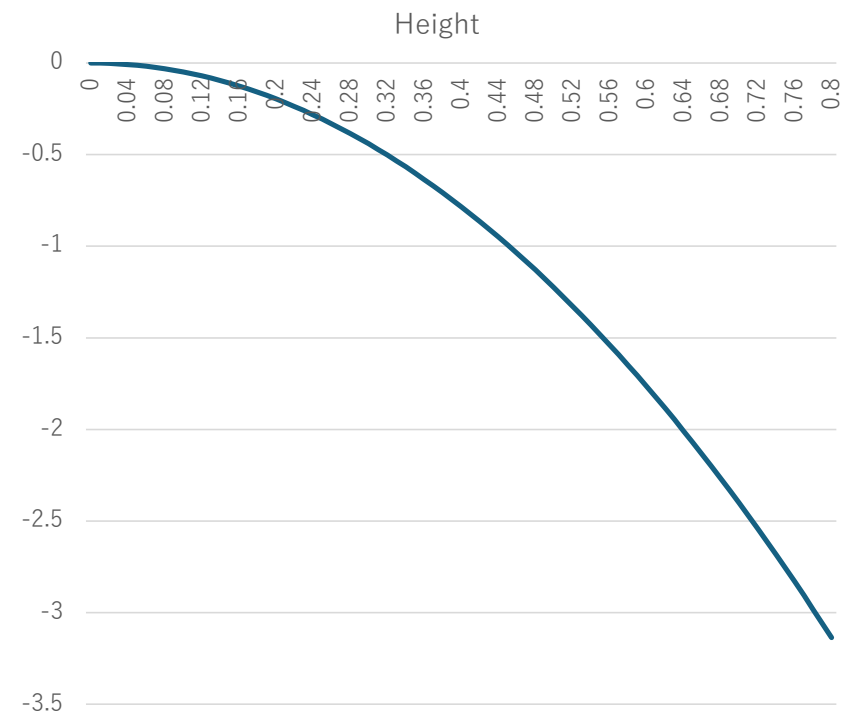
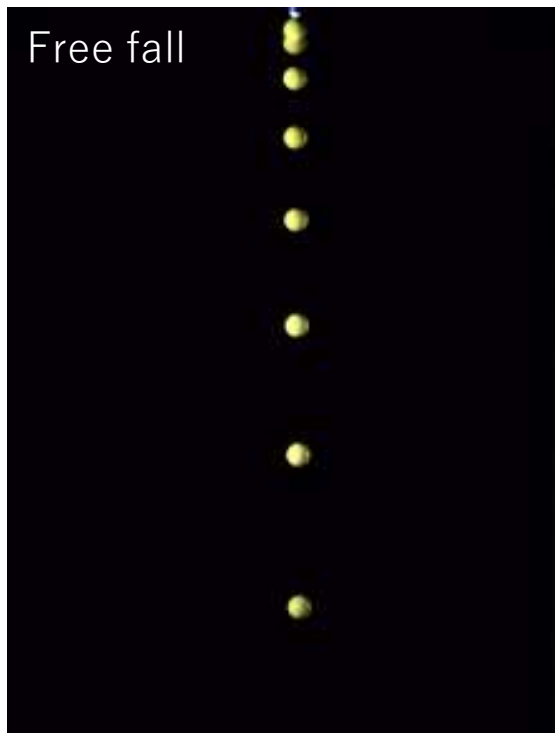
機械学習とは？

数理モデルにおける**調整可能なパラメータ**を、
何らかの基準をもとに自動的に最適化する技術

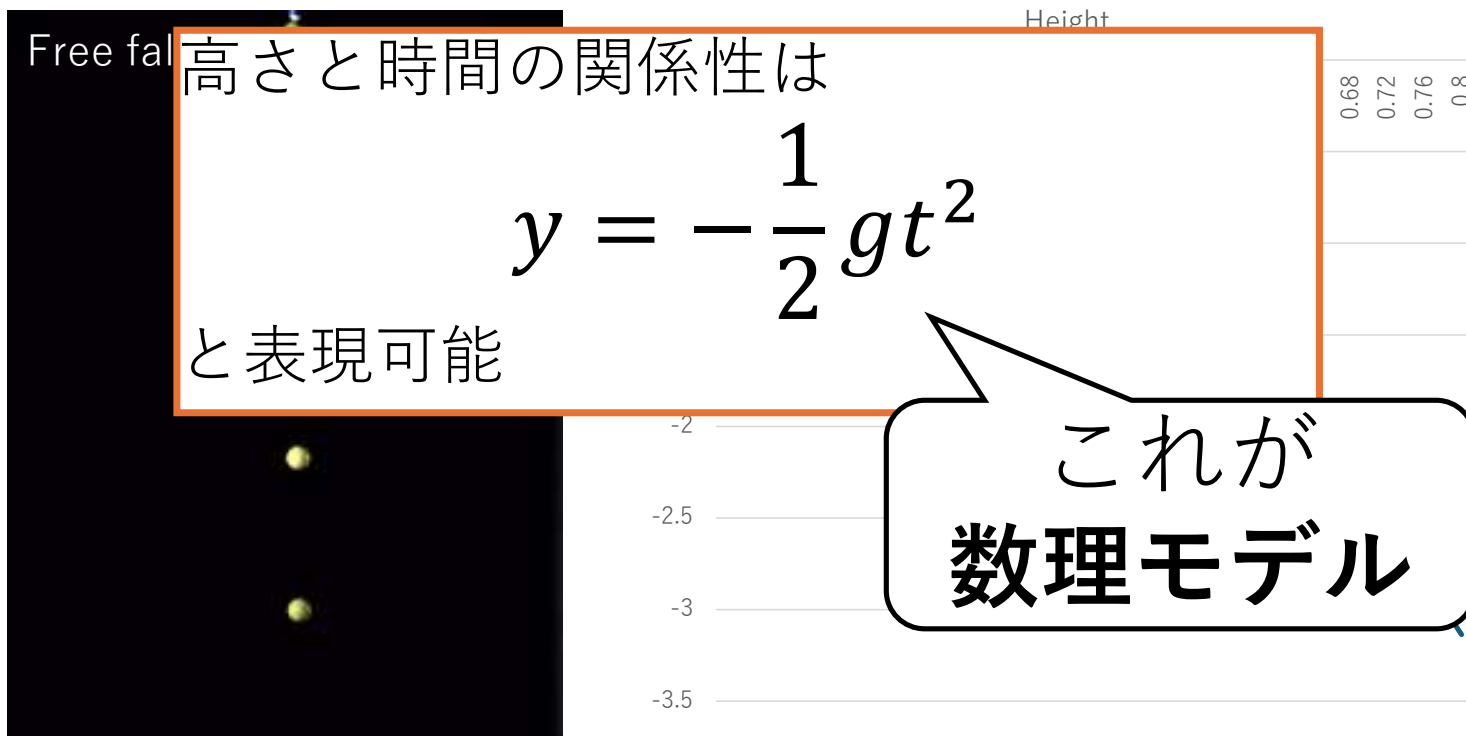
数理モデル：物理現象における複数の物理量や
情報処理における入出力の
関係を数式で表現したもの

パラメータ：モデルの振る舞いを変更できる変数

自由落下について考える



自由落下について考える



数理モデルの振る舞い

数理モデルは、

「その振る舞いを変えることのできる変数」
を持っていることがある。

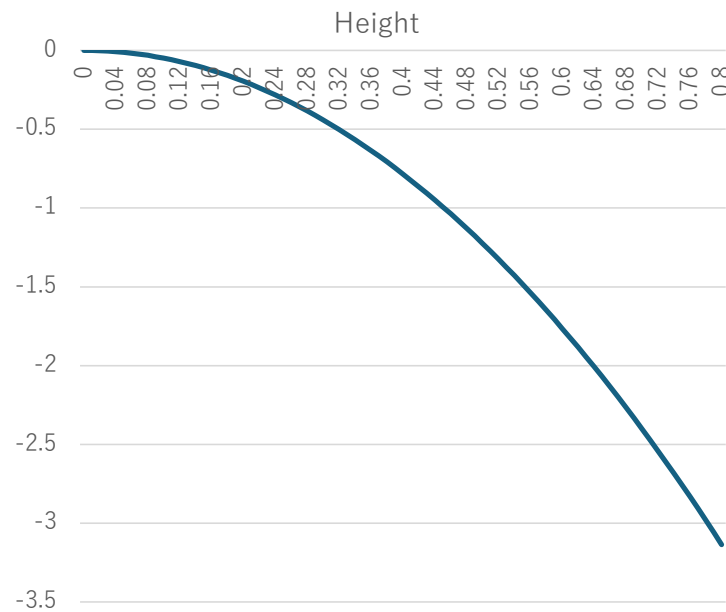
例)

$$y = -\frac{1}{2}gt^2$$

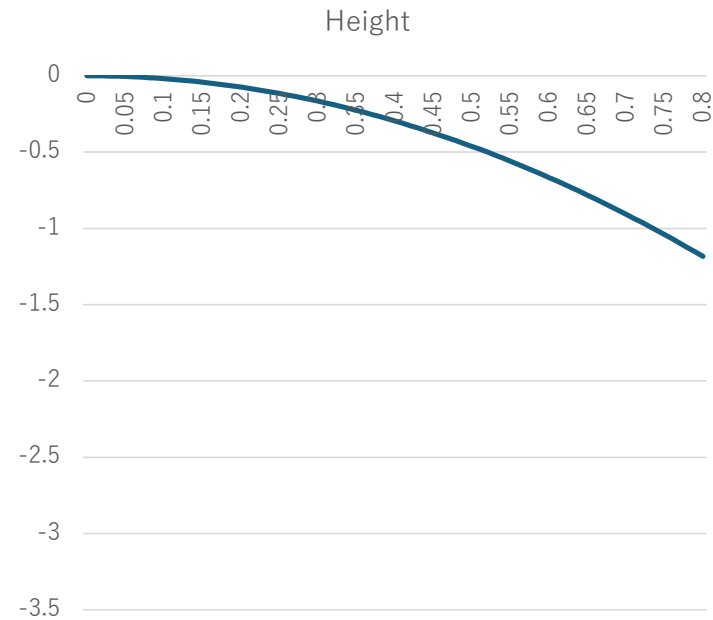
の g はモデルの振る舞いを変更できる

具体例

$g = 9.8$ (Earth)



$g = 3.7$ (Mars)



モデルの振る舞いを変更できる変数 = パラメータ

パラメータの最適値

地球上における g の値は $9.8 \text{ (m/s}^2\text{)}$

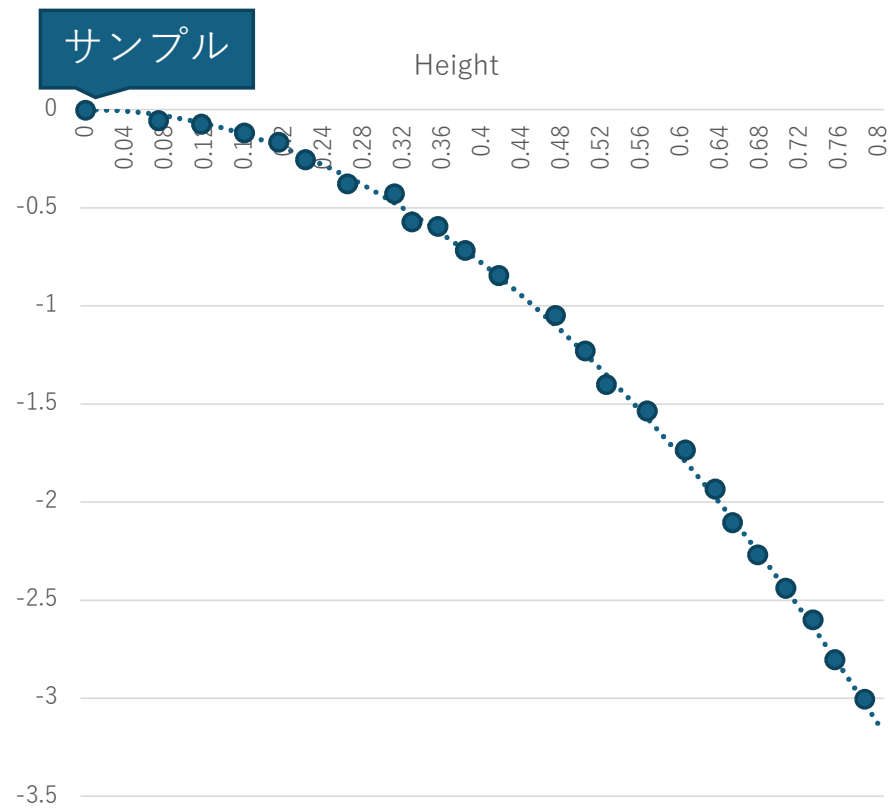
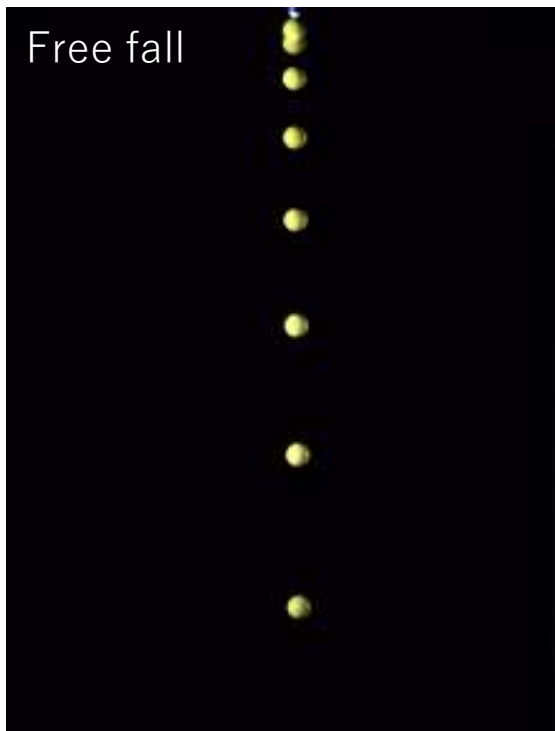
では火星では？

→ある星における g の値を知らない場合、
どのようにこれを推定すればいい？

→実験と観察

実験と観察

$$y = -\frac{1}{2}g_mt^2$$



実験と観察

$$y = -\frac{1}{2}g_mt^2$$

Free fall

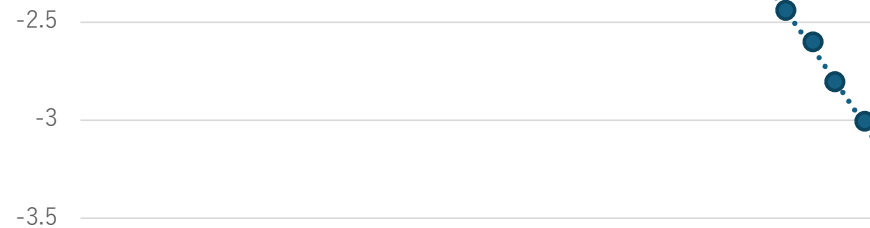
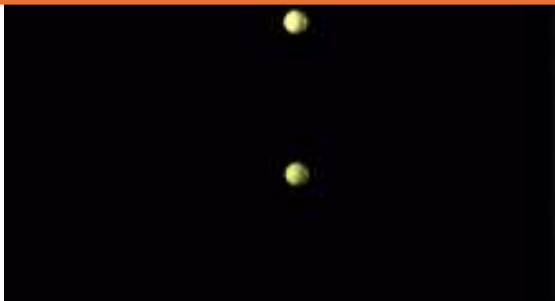


サンプル

Height



何を基準に最適な値 g_m を見つければよいか？
→このケースだと「誤差」がキーに
誤差が小さい g_m が最適だろう



誤差の計算

$$y = -\frac{1}{2}g_mt^2$$

サンプル

$$s_1, s_2, s_3, \dots, s_n = (t_1, y_1), (t_2, y_2), (t_3, y_3), \dots, (t_n, y_n)$$

あるパラメータ g_m を用いたときの t_i における
推定値を $\hat{y}_i$ とする。

損失関数

この時、

$$\sum_{i=1}^n \frac{1}{2} (y_i - \hat{y}_i)^2 = \sum_{i=1}^n \frac{1}{2} \left(y_i - \left(-\frac{1}{2} g_m t_i^2 \right) \right)^2$$

が最小になる g_m を見つければよい

誤差の最小化

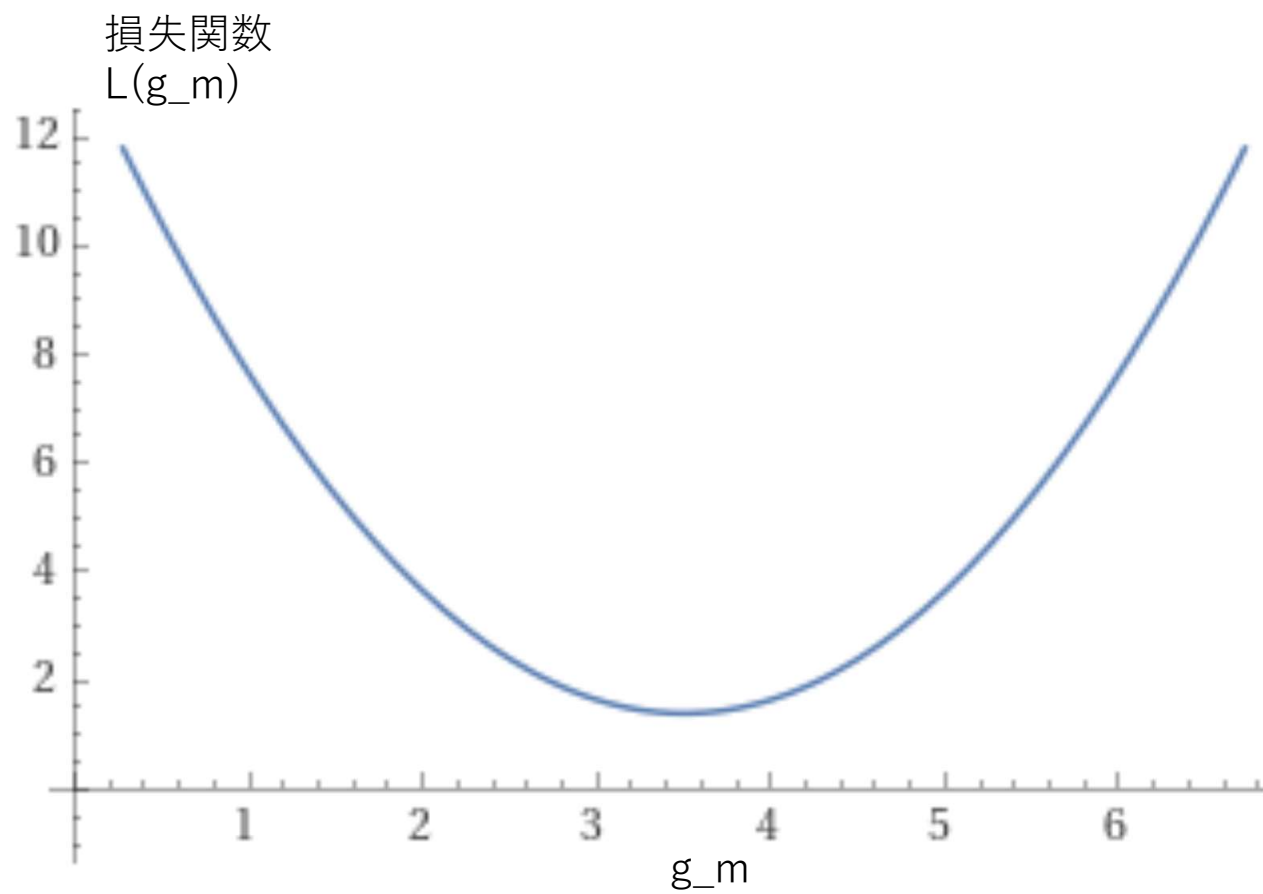
$$\sum_{i=1}^n \frac{1}{2} \left(y_i - \left(-\frac{1}{2} g_m t_i^2 \right) \right)^2$$

t_i, y_i は定数、上記式は g_m の2次式（下に突）
→ 最適な g_m は代数的に計算可能

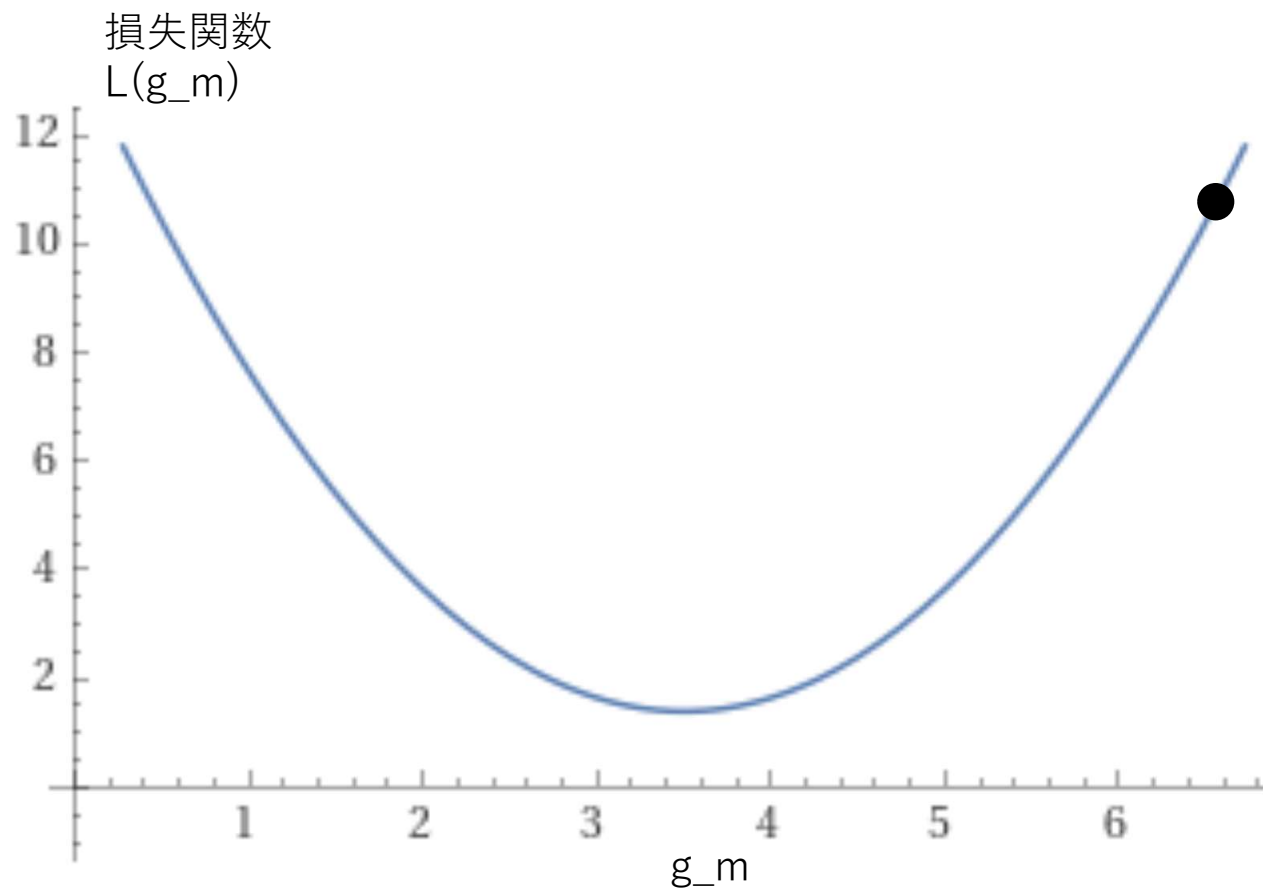
誤差関数がとても複雑になってしまった場合
（パラメータが複数、n次式、log 等を含む
etc）

微分可能な関数でさえあれば、
最急降下法 を利用可能

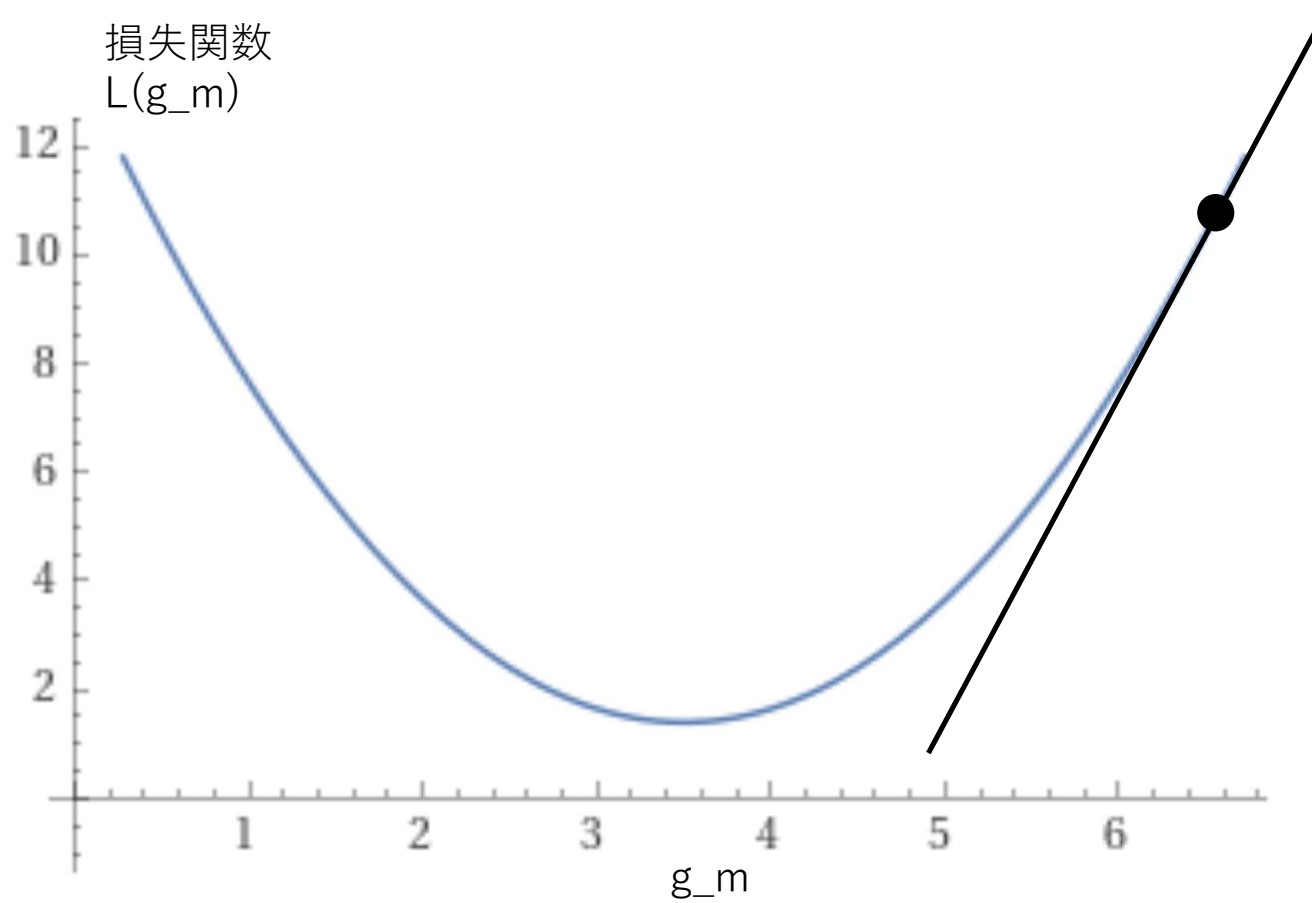
最急降下法のイメージ



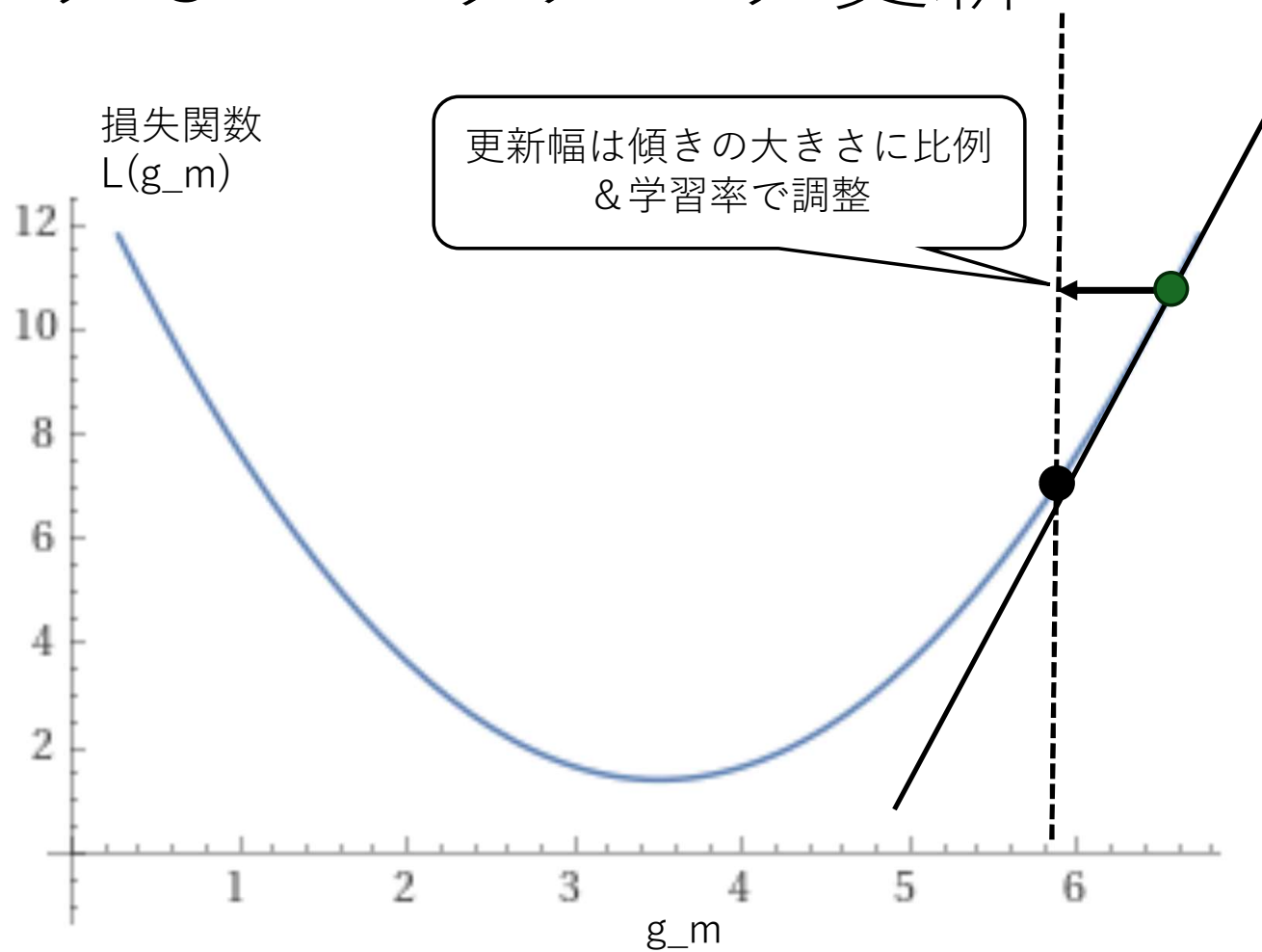
ステップ1 初期値の設定



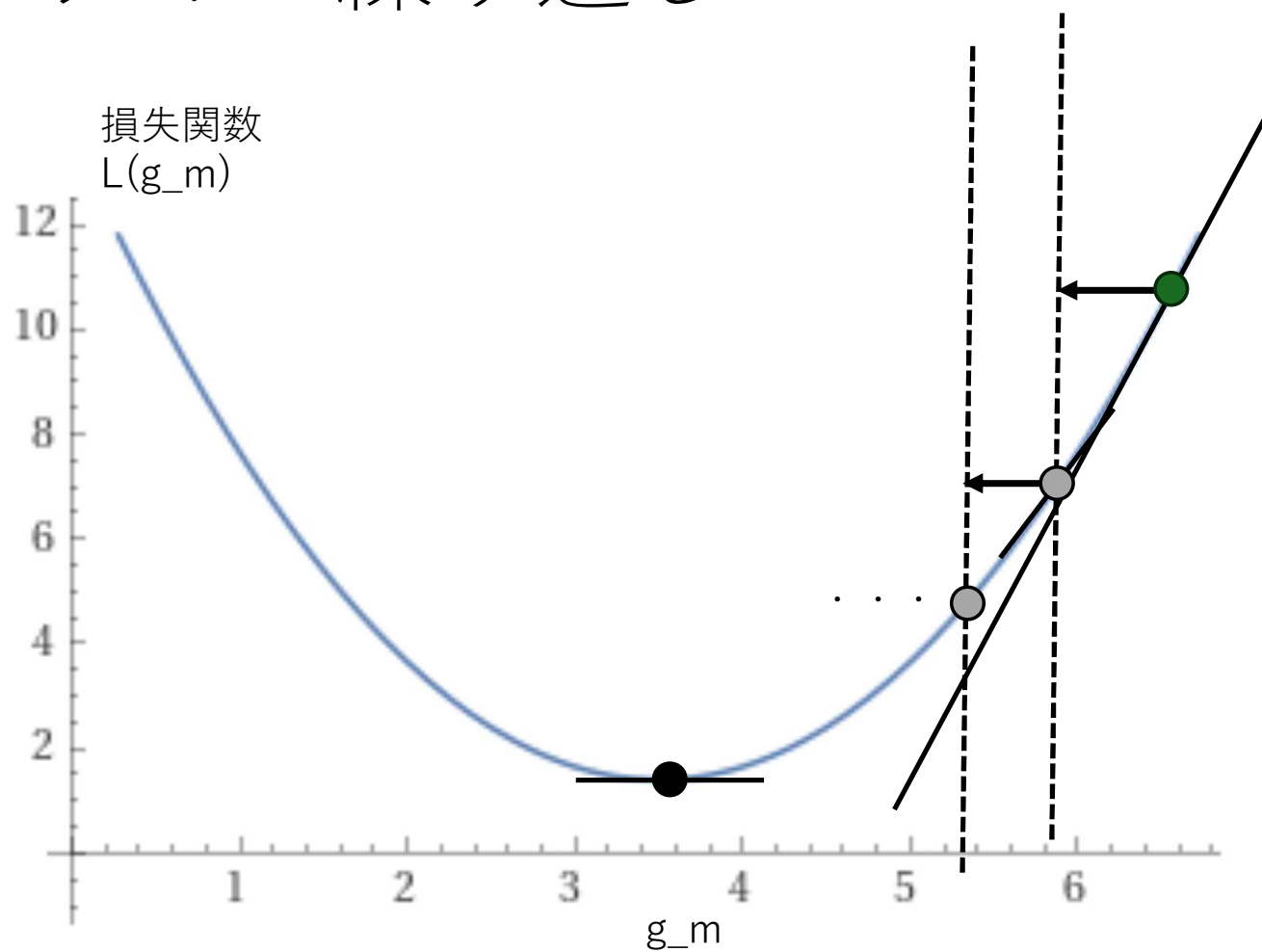
ステップ2 微分



ステップ3 パラメータ更新



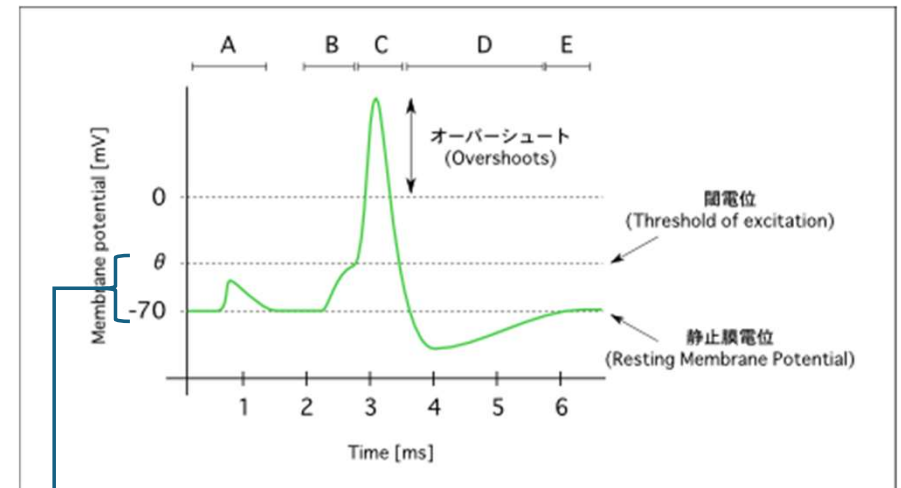
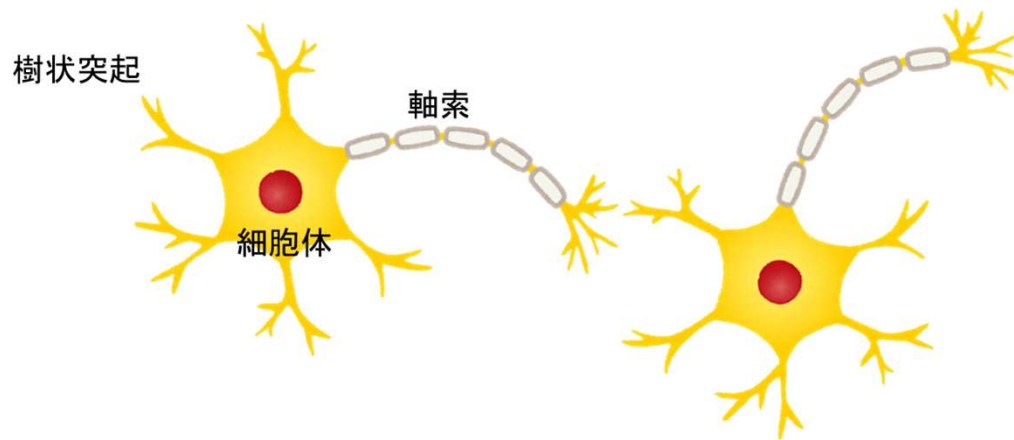
ステップ4 繰り返し



様々な数理モデルが存在するが、
本授業ではニューラルネットワークを

- 脳内のニューロンの活動を模擬することで、
高い表現能力を達成した数理モデルの一種
 - 多くの研究がなされており、
 - パーセプトロン：第一次ニューラルネットブーム
 - 誤差逆伝搬法：第二次ニューラルネットブーム、ノーベル賞2024年
 - 畳み込み：第三次ニューラルネットブーム
 - トランスフォーマーとLLM：第四次？
- といった技術が開発、現在最も性能の良いモデルの一つ

ニューロン再び



$$y = f(\sum w_i x_i - \theta)$$

y : output

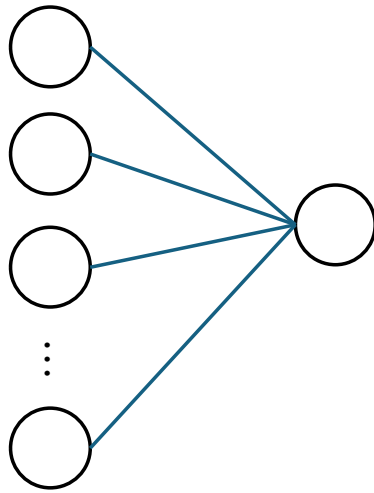
x_i : i -th input (0 or 1)

w_i : i -th synaps weights

θ : firing bias

$f(x)$: step func

形式ニューロン

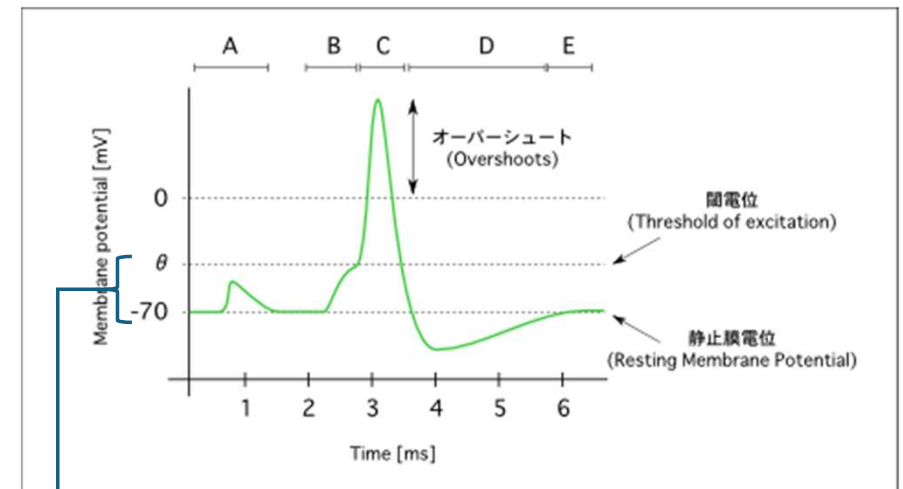


$$y = f(\sum w_i x_i - \theta)$$

y : output

x_i : i -th input ~~(0 or 1)~~

w_i : i -th synaps weights



θ : firing bias

$f(x)$: ~~step func~~ Activation func

形式ニューロン

形式ニューロンは脳ニューロンを参考になっているが、完全な再現はしていない。

- 入力は0/1ではなく実数値
 - 活性化関数がステップ関数ではない
 - そもそも「発火」の概念がない
 - 時間軸的な内部電位の変化を考慮しない
 - シナプス間での神経伝達物質のやり取りを考慮しない
- など。

形式ニューロンの場合，時間軸方向へ情報を圧縮をしている。

入力 = × 発火しているか否か (0/1)

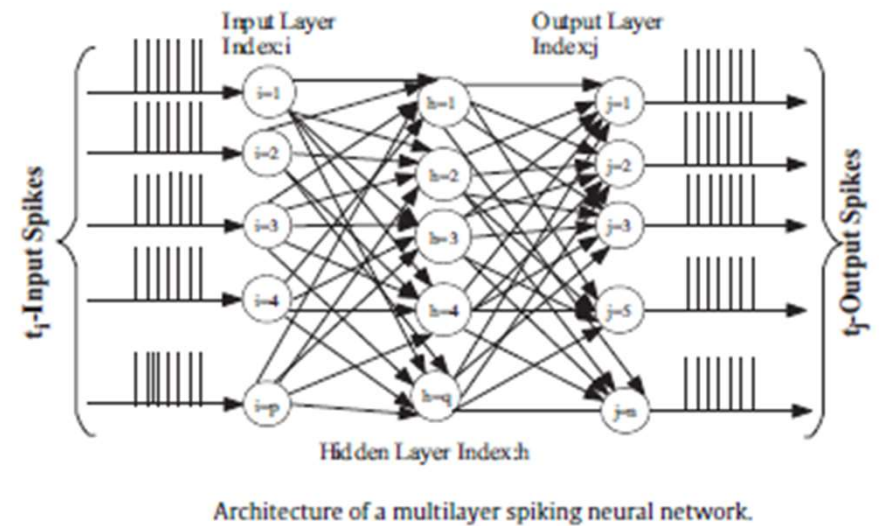
○ 発火頻度 (実数値)

→ 計算量の削減を達成

ちなみに,

Spiking Neural Networks

時系列に従った発火を考慮する
ニューラルネットワーク



ただし, 「数理モデル」というよりは,
脳の「シミュレーション」の意味合いが強い。
脳内の機構解明に関する研究でよく利用

形式ニューロンの限界と発展

$$y = f(\Sigma w_i x_i - \theta)$$

y : output

x_i : i - th input

w_i : i - th synaps weights

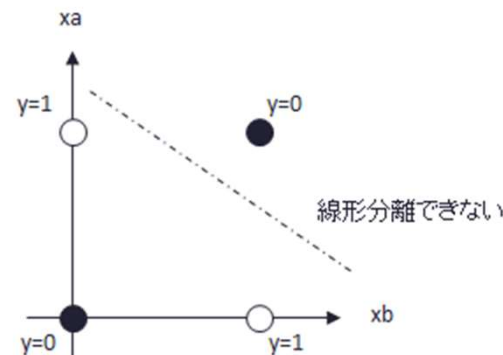
θ : firing bias

$f(x)$: Activation func

形式ニューロンって、
結局直線しか表現できていないのでは？

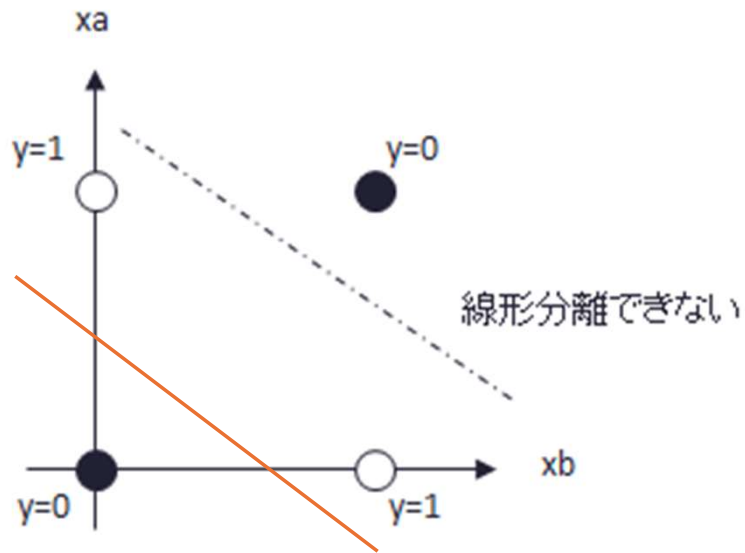
XOR問題を解決できない

→ 第一次ニューラルネット
ブームの終焉



マービン・ミンスキー

XOR問題を解くために



線形分離不可能な問題は、
線 1 本では絶対に解くことができない

→ **線を 2 本使えばいいじゃない。**

= ニューロン 2 つ使う

2 つのニューロンの情報を
どこかで「統合」しなくてはいけない

→ 統合するためのニューロンを用意

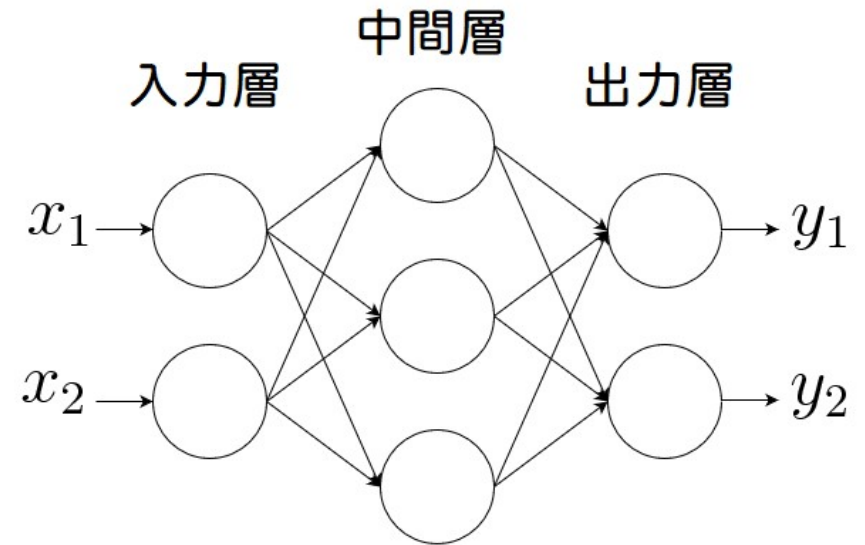
= **多層化**

多層ニューラルネット

中間層の各ニューロンが
作成した線（曲面）を
出力層で組み合わせることで
複雑な関数を表現.

中間層のニューロンが無限に存在すれば
ありとあらゆる関数を表現可能 ということが数学的に証明済み

入出力のペアから中間層の重みをどう学習させるか？
→誤差逆伝搬法



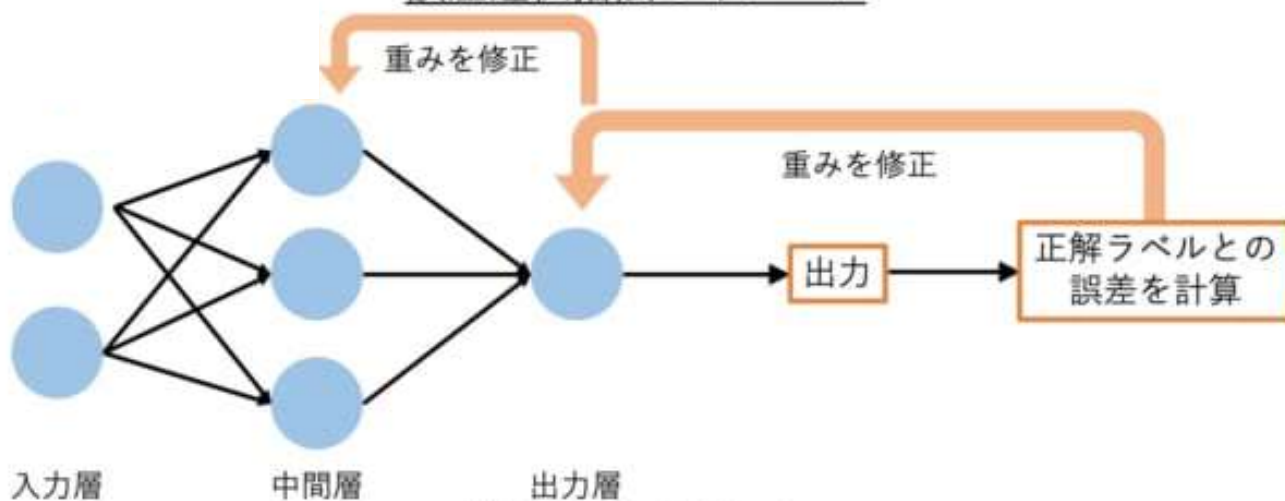
誤差逆伝搬法

誤差逆伝播法（バックプロパゲーション）

誤差逆伝播法

= ネットワークの出力と正解ラベルとの誤差から各ニューロンの重みを修正する

誤差逆伝播法のイメージ



ちなみに、 確率勾配降下法でも多層NNを学習可能

確率的勾配降下法

13の言語版

ページ ノート

閲覧 編集 履歴表示 ツール

出典: フリー百科事典『ウィキペディア (Wikipedia)』

確率的勾配降下法（かくりつてきこうばいこうかほう、英: stochastic gradient descent, SGD）は、連続最適化問題に対する勾配法の乱択アルゴリズム。バッチ学習である最急降下法をオンライン学習に改良したアルゴリズムである。目的関数が微分可能な和の形であることを必要とする。

背景

下記の和の形の目的関数を最小化問題を扱う。

$$Q(w) = \sum_{i=1}^n Q_i(w)$$

パラメータ w^* は $Q(w)$ を最小化するように推定する。典型的には、 Q_i は i 番目の訓練データ。

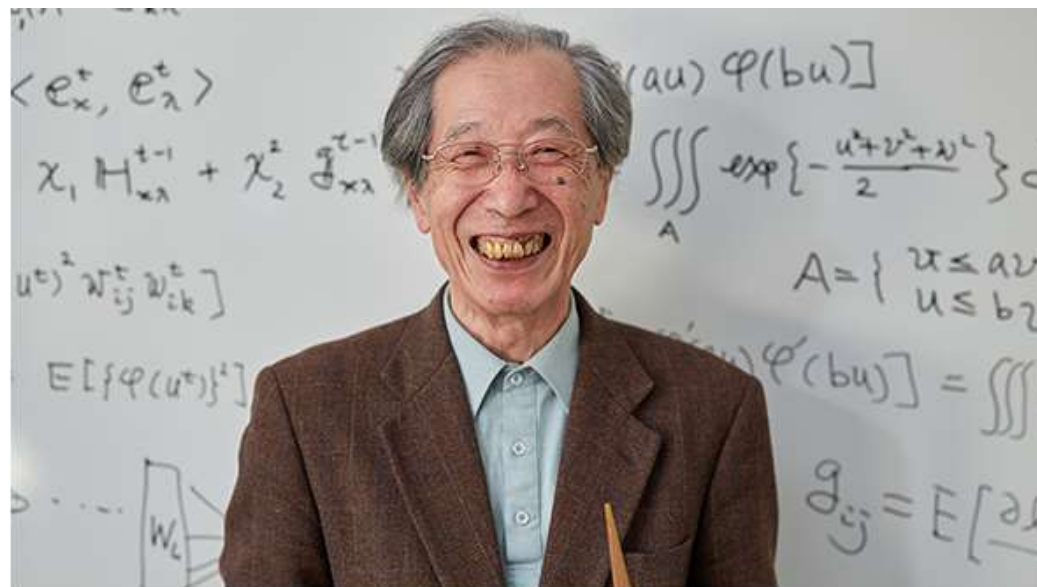
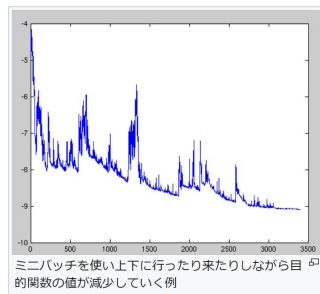
古典的な統計学において、和の最小化問題は、最小二乗問題や最尤推定問題などにあらわれる。一般的なケースでは、和を最小化する推定量はM推定量と呼ぶ。しかしながら、Thomas S. Fergusonの例^[1]などで示されるように、いくつかの最尤推定の問題において、最小解ではなく局所解を要求するだけでも制限が厳しすぎると長い間認識され続けてきた。それゆえ、現代の統計理論家は最尤関数の停留点(微分がゼロになる点)を考慮する事が多くなった。

和の最小化問題は経験損失最小化（英語版）の問題にも現れる。 $Q_i(w)$ の値が i 番目の訓練データであるならば、 $Q(w)$ が経験損失である。

上記の関数 Q を最小化する際、標準的な最急降下法（バッチ学習）では、下記の反復を繰り返す。

$$w := w - \eta \nabla Q(w) = w - \eta \sum_{i=1}^n \nabla Q_i(w)$$

η はステップサイズと呼ばれる。機械学習においては学習率（英語版）とも呼ばれる。



甘利俊一先生

層数と表現可能な関数の複雑さ

もっと多層にしていったらどうなるんだろう？

中間層が一つの場合，

→出力層は線や曲面の重ね合わせで関数を表現

中間層が二つの場合，

→出力層は「線や曲面の重ね合わせ」の重ね合わせで関数を…

中間層がNつの場合，

→出力層は「「「「…」

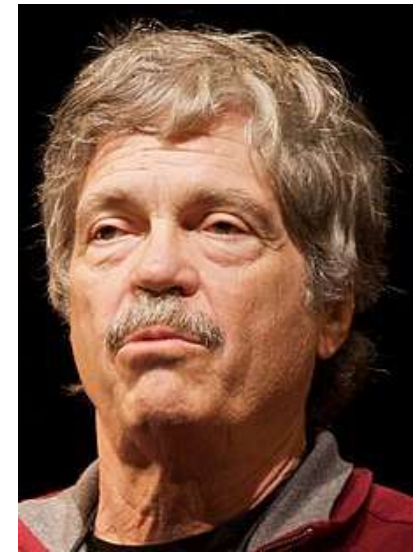
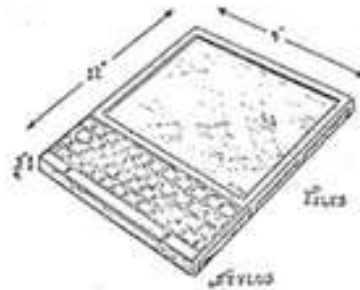
基本的に，層数を増やすと少数のニューロンでも表現可能な関数は爆発的に増加するはず。

**問題は
計算量**

2011年ごろの時代背景と 深層ニューラルネット

実は、ソフトウェアやアルゴリズム、アイデアの発展は、
ハードウェアの進歩よりもほぼほぼ先行する。

例) ダイナブック構想とiPad



iPad(2010年発表)とほぼ同じもののアイデアは1970年代に初出

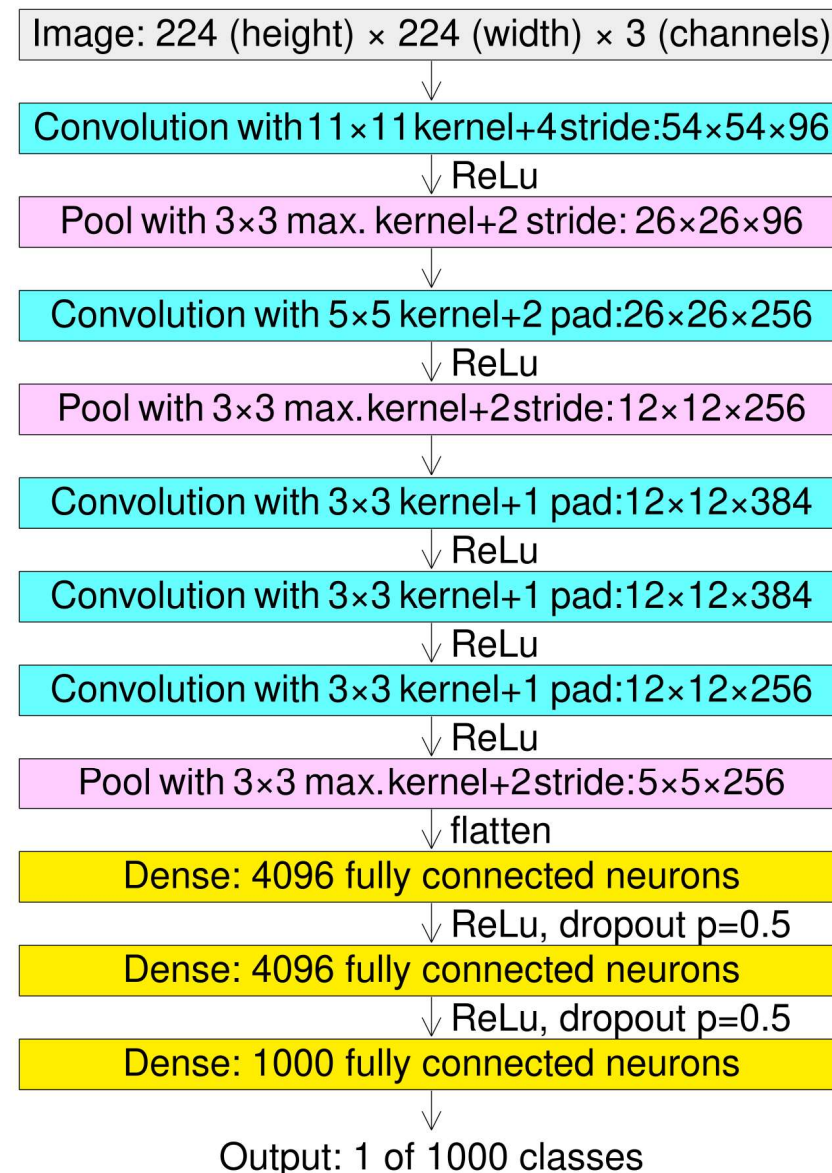
AlexNetの登場

一般物体認識課題のコンペに登場,
Hand-Engineeredな特徴量を使った
コンペ相手をごぼう抜き

第三次ニューラルネットブームへ



AlexNet



結局深層ニューラルネットって何なん？

浅層なニューラルネットと比較して
層数が比較的多いニューラルネットの総称

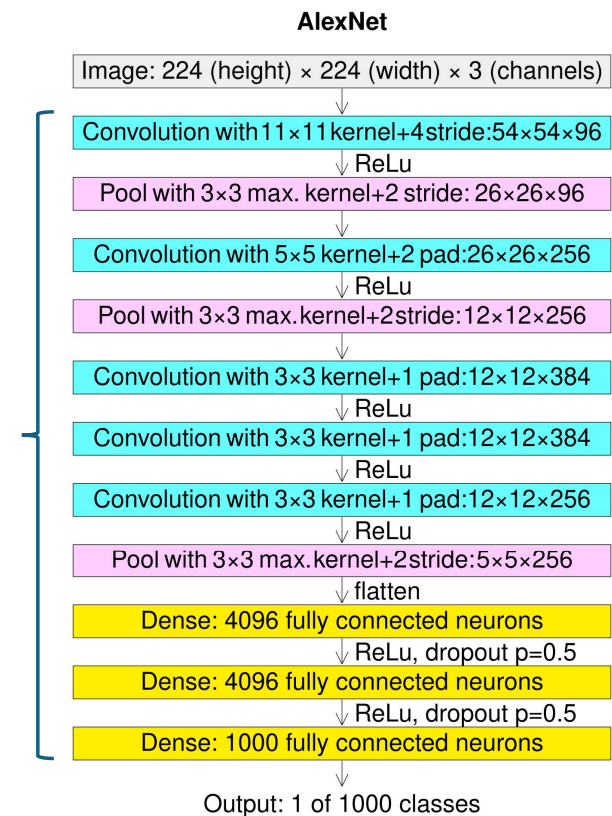
としか言いようがない。

具体例を見ていくと、

- 畳み込みニューラルネット
- リカレントニューラルネット
- トランスフォーマー

等々

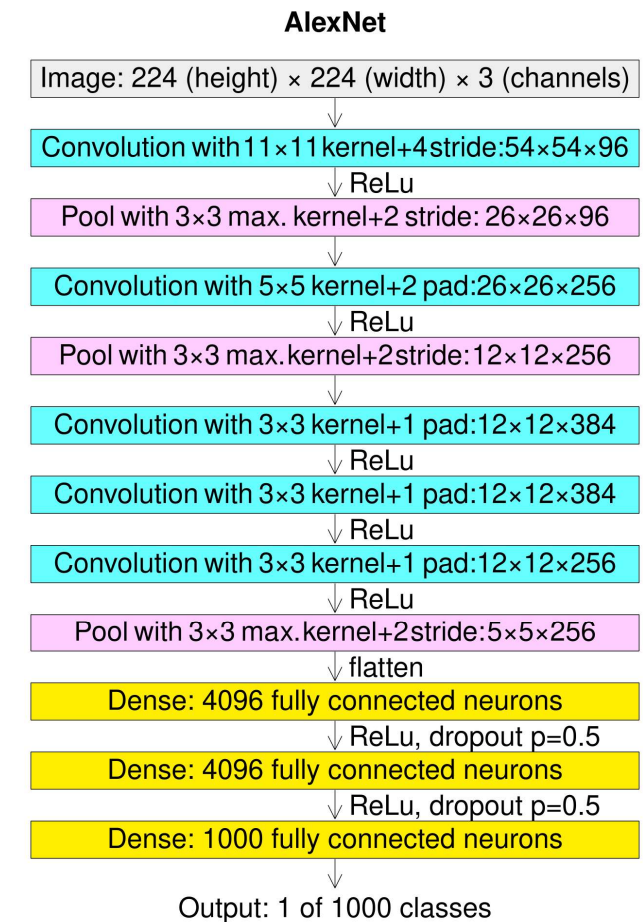
あくまで感覚的だけど、
5層くらいあれば
「深層学習」感がある



AlexNetの構成要素

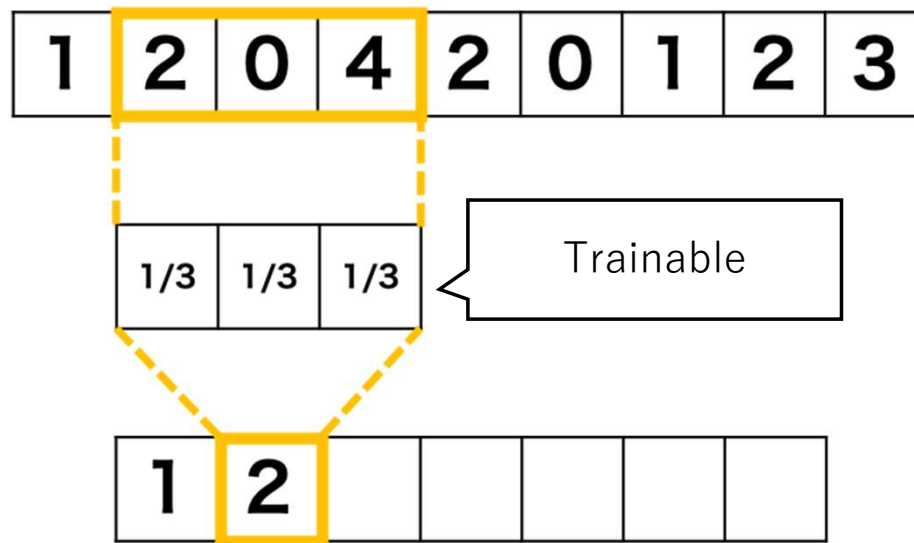
大きく分けると

- Convolution (畳み込み)
 - Pool
 - Dence (全結合)
 - Flatten
- の 4 つ



畳み込みニューラルネット

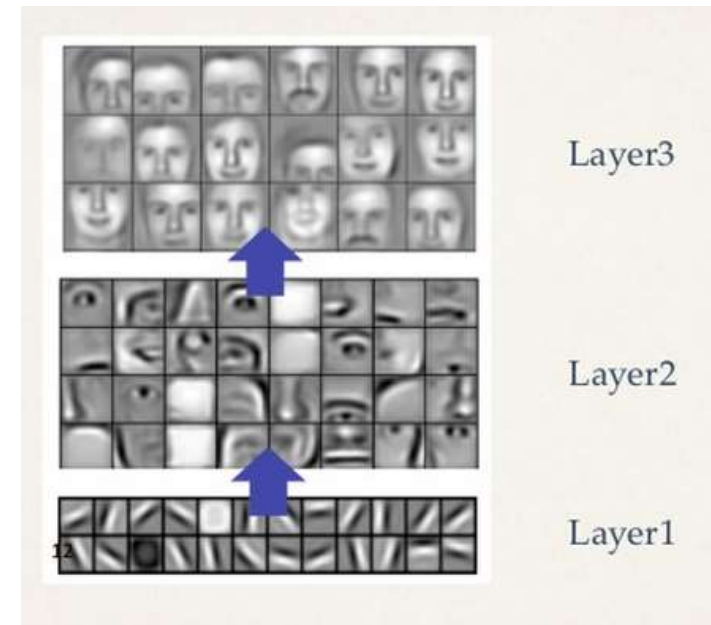
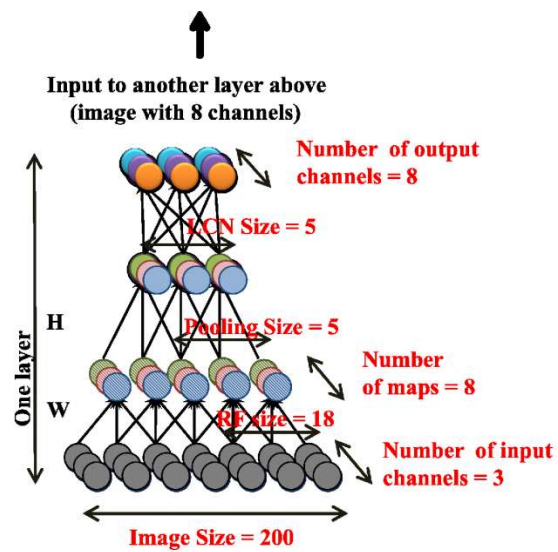
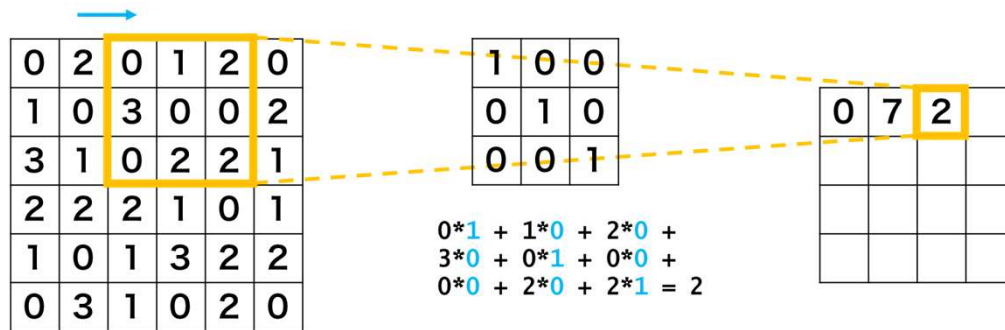
「畳み込み」と呼ばれる、
学習可能な局所フィルタを行うニューロンを持つNN



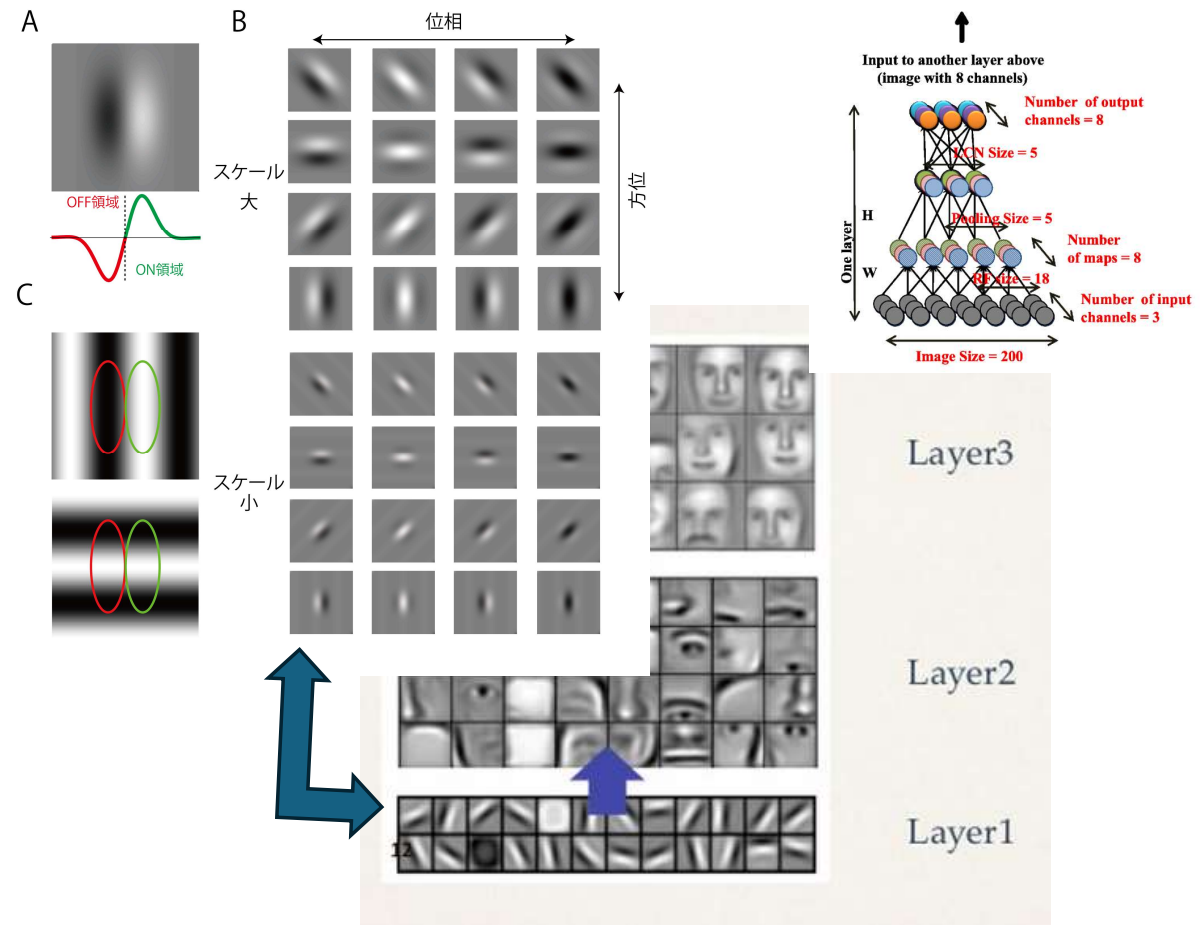
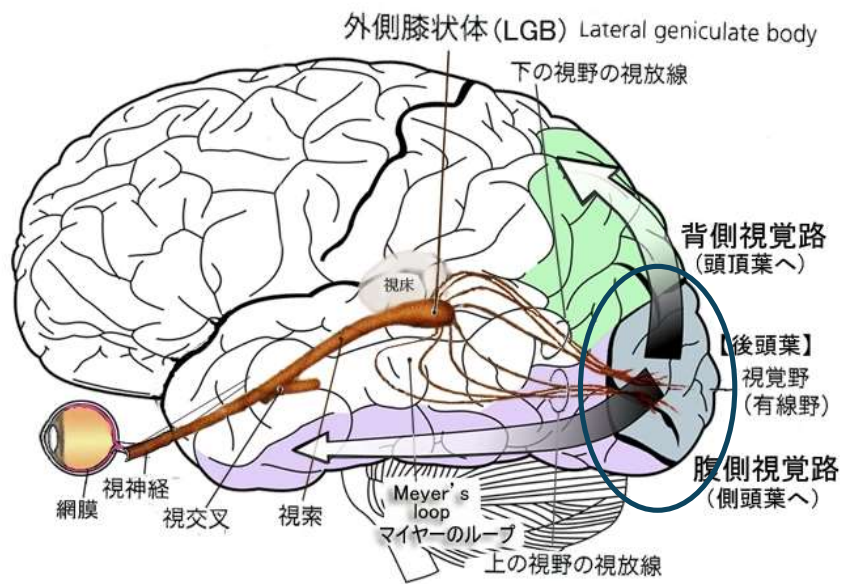
全結合とは異なり、ある入力
と重みの対応付けが無い

一次元信号の場合、
適応フィルタと概念に近い

畳み込みニューラルネット



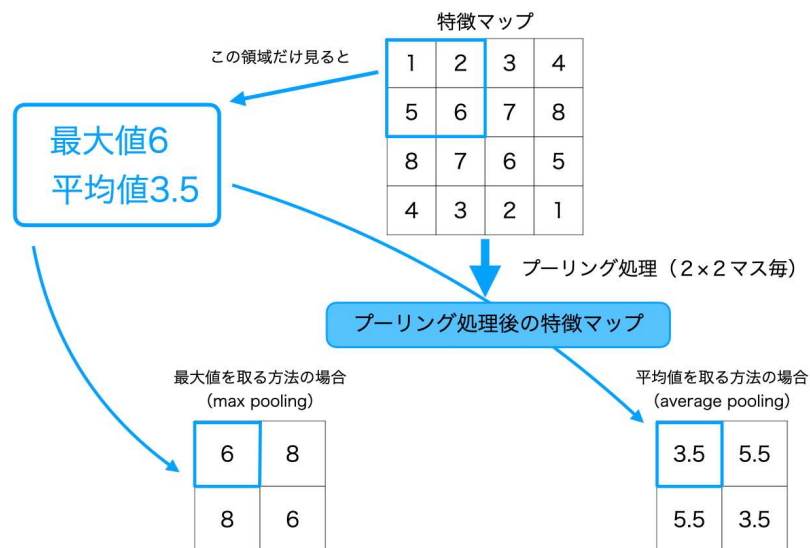
畳み込みニューラルネットとV1



プーリング層

下位の層で抽出してきたベクトル，テンソルに対して，ある窓の範囲内の値の最大値や平均値をとる操作のこと

プーリング処理



位置ずれに対する頑健性の向上が目的

全結合層とFlatten

<全結合層>

一般的な単層ニューラルネットと同一の物.

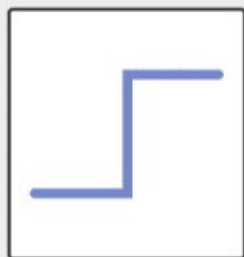
<Flatten>

抽出されたベクトル，テンソルを一次元ベクトルに変換する

時系列信号や画像から抽出された情報は
その並び順に意味がある状態だが，
この操作以降並び順に意味がなくなる.

活性化関数

01



Step Function
ステップ関数

02



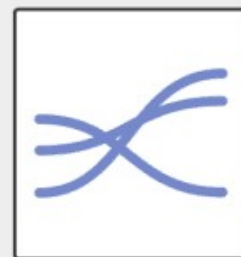
Sigmoid Function
シグモイド関数

03



ReLU Function
ReLU関数

04



Softmax Function
Softmax関数

05



Identity Function
恒等関数

損失関数

モデル全体を最適化するうえでの指標

この関数が最小化/最大化するようにパラメータを調整

- 平均二乗誤差

$$E = \frac{1}{n} \sum_k^n (T_k - y_k)^2$$

T_k : 真値 (実数値)

- 交差エントロピー

$$E = -\sum_k^n T_k \log(y_k)$$

T_k : 真値 (0/1)

正則化

基本的に、損失関数はその目的に合ったものを使用する。
大目的と同時に小目的を達成したいこともある

→損失関数をブレンドすることで、
複数の目的を達成しよう

小目的としてよく採用されるもの
「過剰適合を避けたい」

→「正則化項」をブレンドして
小目的も達成しよう



過剰適合時の重みの特徴

過剰適合時のような複雑な関数は、
極端な重みを採用しないと
再現できない

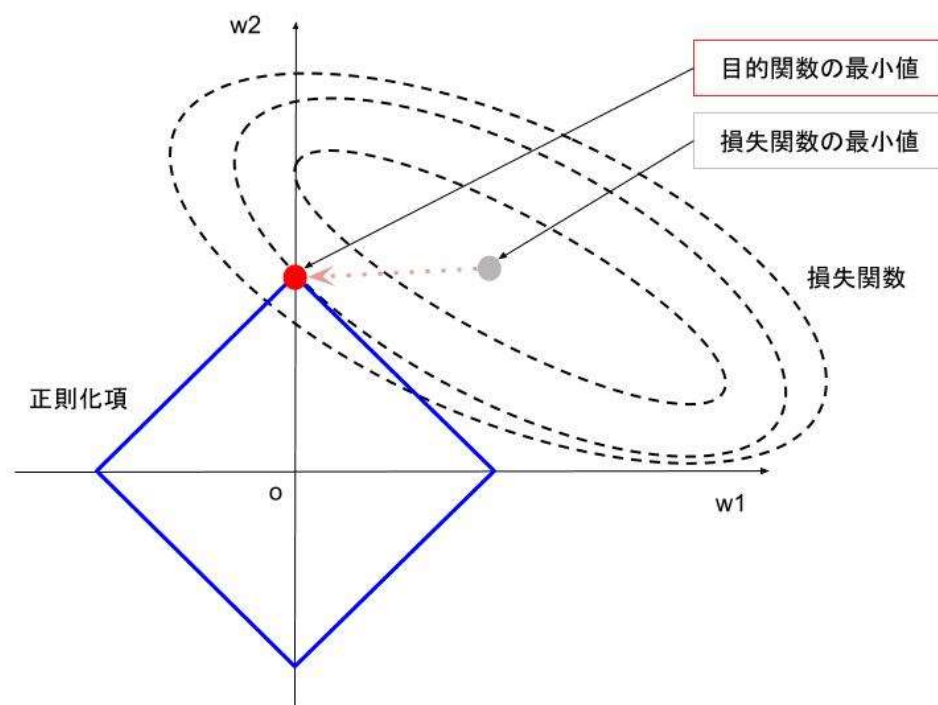
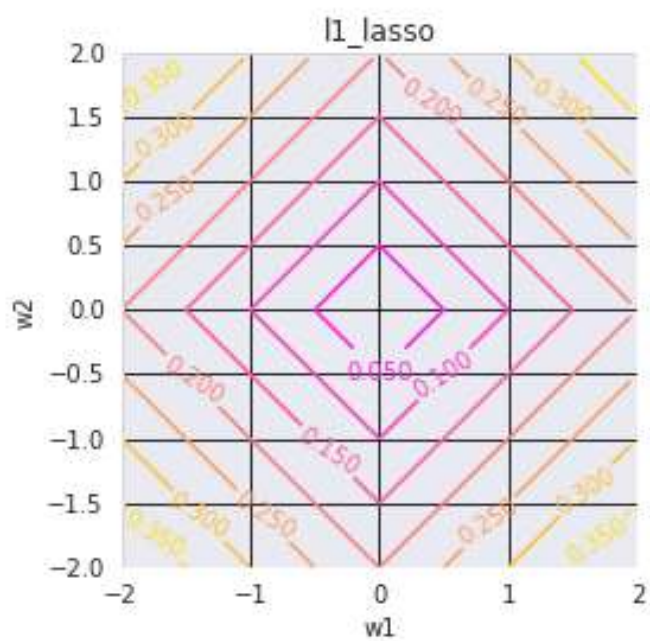


→逆説的に、極端な重み（＝重みの絶対値が大きい）
にならないよう調整すれば過剰適合は避けられる

- L1ノルム
→重みの絶対値の平均
- L2ノルム
→重みの二乗平均根号

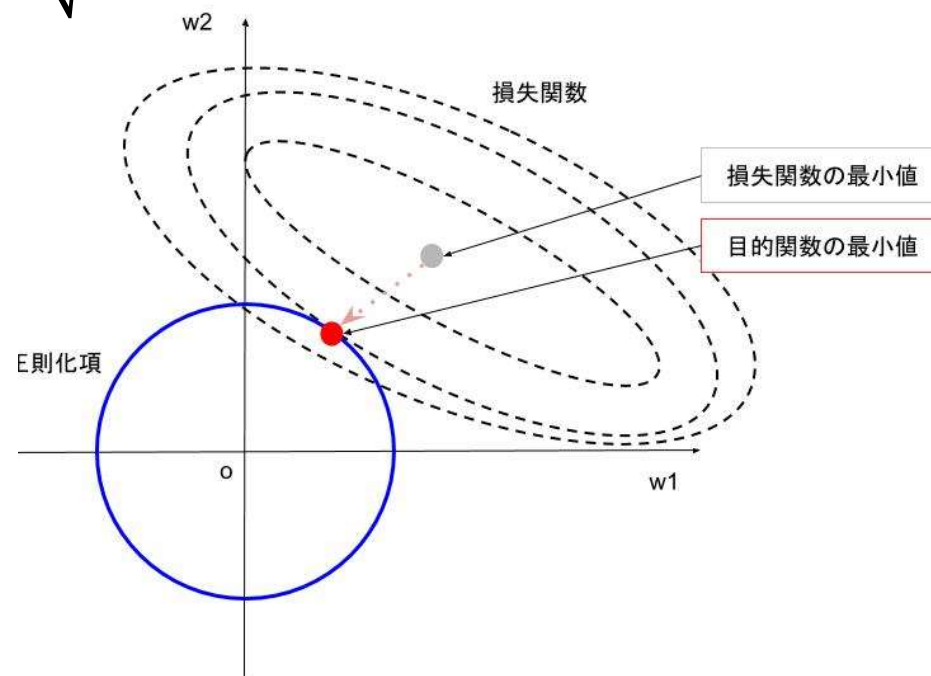
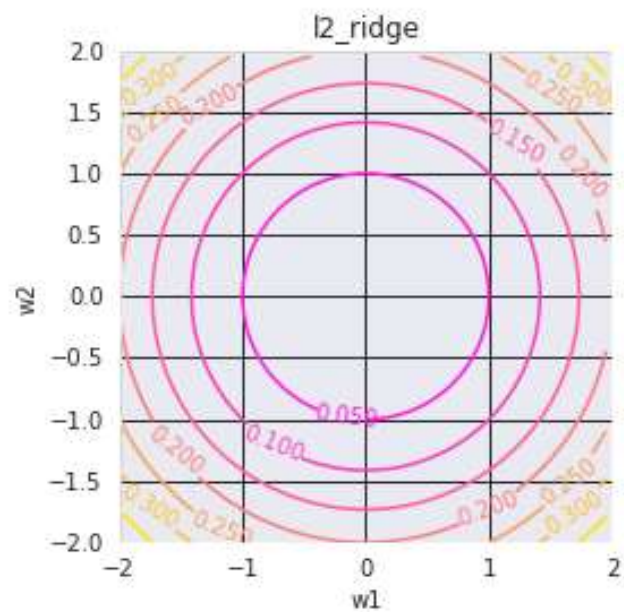
L1ノルム

$$L_1 = \sum_k |w_k|$$



L2ノルム

$$L_2 = \sqrt{\sum_k w_k^2}$$



その他の過剰適合回避手法

そもそも、過剰適合は
モデルのパラメータ数に対してサンプルが少ないから発生
→サンプル数の追加

学習回数が多くなると重みは極端な値にならない
(重みの更新量は学習率で制御されているので)
→そもそも学習を途中でやめてしまう
Early Stopping

補助損失

例えば，生体信号のデータセットA，Bがあるとする．
微妙に信号特性が異なり，2つを混ぜて訓練するのが難しい．

→（ステージ判定のための損失）
 -（データセット判定のための損失）

この部分が補助損失

データセットを判定するための損失を最大化

→データセットの信号特性の違いが

抽出されない（無視される）方向に学習が進む